



University of
Nottingham
UK | CHINA | MALAYSIA

COMP2001/2011: Artificial Intelligence Methods

Lecture 5 – Evolutionary Algorithms I

Prof Ender Ozcan





Learning Outcomes

- Be able to identify population-based metaheuristics
- Understand the underlying analogy for Evolutionary Algorithms
- Be able to identify different types of Evolutionary Algorithms
- Understand the basic components of Genetic Algorithms as well as Memetic Algorithms, including mutation, crossover, parent/mate selection, replacement, and how each operates – learning the jargon
- Be able to design and apply a Genetic/Memetic Algorithm to an “unseen” problem





Outline

- Analogy of Evolution
- Evolutionary Algorithms
- Genetic Algorithms (GAs)
- Memetic Algorithms (MAs)





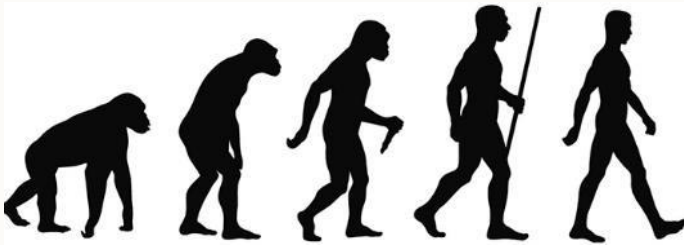
Evolutionary Algorithms

Evolutionary is Revolutionary



Evolutionary is Revolutionary

- Nature as a Problem Solver: 4.55 Billion years of evolution can't be wrong.



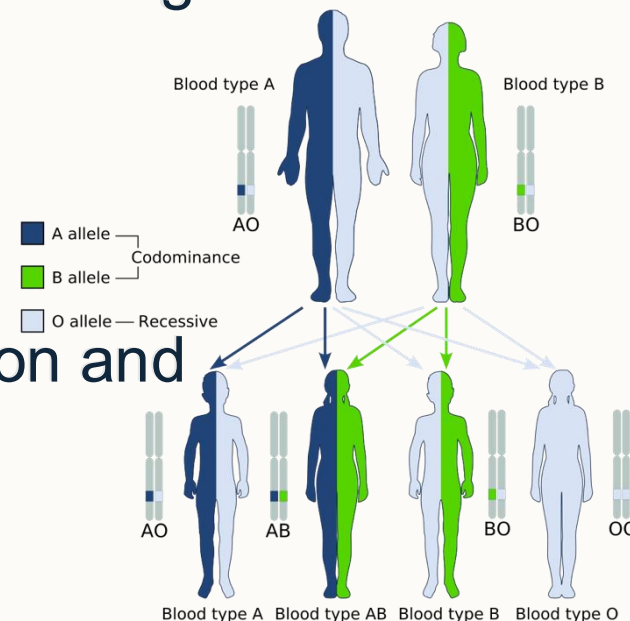
Evolution: Gradual change in the inherited characteristics of a population of animals or plants over successive generations

- Beauty-of-nature argument: Complexity achieved in *short* time in nature.
- Can we solve complex problems as quickly and reliably on a computer?



Evolution at Work

- Heritable characteristics or heritable traits, e.g., the blood types are passed from one generation to the next via DNA (a molecule that encodes genetic information)
- Change or genetic variation comes from:
 - Mutations: changes in the DNA sequence,
 - Crossover: reshuffling of genes through sexual reproduction and migration between populations
- Evolution is driven by natural selection – survival of the fittest
- Genetic variations that enhance survival and reproduction become and remain more common in successive generations of a population





A famous example – peppered moth evolution

white/black-bodied peppered moth



- Before the Industrial Revolution, the black peppered moth was rare.
 - The frequency of the dark allele was about 0.01%
- The rapid industrialisation and rampant coal use coated the British trees in a layer of soot
- By the mid-19th century, the number of dark-coloured moths had risen noticeably, and by 1895, the frequency was 98% in Manchester



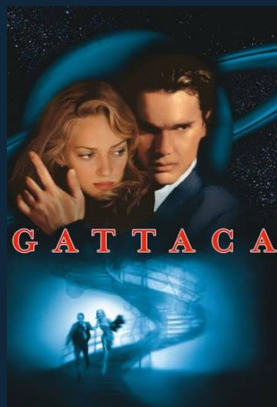
Evolutionary Algorithms (EAs)

- EAs simulate natural evolution (Darwinian Evolution) of individual structures at the genetic level using the idea of **survival of the fittest** via processes of **selection, mutation, and reproduction (recombination)**
- An **individual (chromosome)** represents a candidate solution for the problem at hand. (e.g., $\langle 2 \ 1 \ 3 \ 4 \rangle$)
- A collection of individuals currently “alive”, called **population** (set of individuals/chromosomes) is evolved from one **generation (iteration)** to another depending on the **fitness** of individuals in a given environment, indicating how fit an individual is, (how close it is to the optimal solution) – objective value. (e.g., $f(\langle 2 \ 1 \ 3 \ 4 \rangle) = 28$)
- **Hope:** Last generation will contain the best solution



Evolutionary Algorithms (EAs) II

- **Genetic Algorithms** (evolves (bit) strings) → Nils Aall Barricelli 1954, Holland 1975
 - Memetic Algorithms → Moscato 1989
- **Evolutionary Programming** (evolves parameters of a program with a fixed structure) → Fogel, Owens, Walsh 1966
- **Evolution Strategies** (vectors of real numbers) → Rechenberg 1973
- **Genetic Programming** (evolves computer programs in tree form) → Koza 1992
 - **Gene Expression Programming** (computer programs of different sizes are encoded in linear chromosomes of fixed length)
 - **Grammatical Evolution** (evolves solutions wrt a specified grammar) → Ryan, Collins and O'Neill 1998



Genetic Algorithms (GAs)





Pseudocode of a Generic Genetic Algorithm

Representation

begin

```
generate initial population; // initialise
calculate fitness values;    // evaluate population
do
{
    perform reproduction;    // select parents (mates)
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
```

end



Basic Components of GAs

- A genetic **representation (encoding)** for candidate solutions (individuals) to the problem at hand
- An **initialisation** scheme to generate the first population (set) of candidate solutions (individuals)
- A **fitness (evaluation) function** that plays the role of the environment, rating the solutions in terms of their fitness
- A scheme for **selecting mates (parents)** for recombination
- **Crossover (recombination)** exchanges genetic material between mates producing **offspring (children)**
- **Mutation** perturbs an individual creating a new one
- **Replacement strategy** to select the surviving individuals for the next generation
- **Termination Criteria**
- Values for various parameters that GA uses (population size, probabilities of applying genetic operators, etc)



Pseudocode of a Generic Genetic Algorithm

Representation

begin

```
generate initial population; // initialise
calculate fitness values;    // evaluate population
do
{
    perform reproduction;    // select parents
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
```

end



GA Components: Representation

- **Haploid structure** is used: Each individual contains one chromosome
- Each individual is evaluated and has an associated **fitness** value
- Chromosomes contain a fixed number of genes: **chromosome length**
- Traditionally **binary encoding** is used for each gene: **Allele** value $\in \{0,1\}$
- A population contains a fixed number of individuals: **population size**
- Each iteration is referred to as **generation**



Pseudocode of a Generic Genetic Algorithm

Representation

begin

```
generate initial population; // initialise
calculate fitness values;    // evaluate population
do
{
    perform reproduction;    // select parents
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
end
```



GA Components: Initialisation

- Random Initialisation
- Population size number of individuals are created randomly
- Each gene at a locus of an individual is assigned an allele value 0 or 1 randomly, decided by flipping a coin (E.g., if the random value is <0.5 , then allele is assigned to 0, otherwise to 1).

Chromosome (individual)

gene

0	1	1	0	1
1	2	3	4	5

locus



Example – MAX-SAT: Initialisation

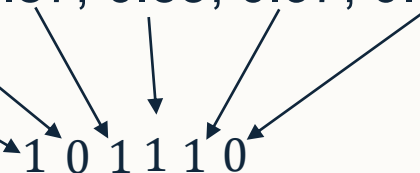
$$\begin{array}{cccccc} 0 & & 1 & & 2 & & 3 & & 4 & & 5 \\ (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \end{array}$$

1: <0.98, 0.85, 0.13, 0.04, 0.47, 0.44>

2: <0.09, 0.63, 0.22, 0.54, 0.07, 0.37>

3: <0.21, 0.03, 0.72, 0.84, 0.19, 0.49>

4: <0.61, 0.43, 0.87, 0.53, 0.97, 0.14>



- Assume population size is 4
- The individual size/chromosome length is 6 (since we have 6 literals: abcdef)
- So, create 4 individuals with 6 genes within their chromosomes, where each allele at a locus is determined randomly (by throwing a random number in $[0,1)$).



Example – MAX-SAT: Initialisation

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \end{array}$$

Chromosome	
i	abcdef
1:	110000
2:	010100
3:	001100
4:	101110

- Assume population size is 4
- The individual size/chromosome length is 6 (since we have 6 literals: abcdef)
- So, create 4 individuals with 6 genes within their chromosomes, where each allele at a locus is determined randomly (by throwing a random number in $[0,1)$).



Pseudocode of a Generic Genetic Algorithm

Representation

begin

```
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
    perform reproduction; // select parents
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
```

end



GA Components – Fitness Calculation

- **Fitness value** indicates
 - how fit the individual is to survive and reproduce under the current conditions
 - How much does the current solution meet the requirements of the objective function
- **Fitness value** is obtained by applying the fitness function to the individual's chromosome (candidate solution) – *genotype* (e.g., 101110) to *phenotype* (e.g., 1) mapping



Example – MAX-SAT: Fitness Calculation

$$\begin{array}{cccccc} 0 & & 1 & & 2 & & 3 & & 4 & & 5 \\ (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \end{array}$$

Chromosome		Unsatisfied clauses	Fitness
i	abcdef		
1	110000		
2	010100		
3	001100		
4	101110		



Example – MAX-SAT: Fitness Calculation

$$\begin{array}{cccccc} 0 & & 1 & & 2 & & 3 & & 4 & & 5 \\ (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \end{array}$$

Chromosome		Unsatisfied	
i	abcdef	clauses	Fitness
1	110000	012345	3
2	010100		
3	001100		
4	101110		



Example – MAX-SAT: Fitness Calculation

$$\begin{array}{cccccc} 0 & & 1 & & 2 & & 3 & & 4 & & 5 \\ (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \end{array}$$

Chromosome		Unsatisfied	
i	abcdef	clauses	Fitness
1	110000	012345	3
2	010100	012345	3
3	001100	012345	2
4	101110	012345	1



Pseudocode of a Generic Genetic Algorithm

Representation

begin

generate initial population; // initialise

calculate fitness values; // evaluate population

do

{

perform reproduction; // select parents

recombine pairs with p_c ; // apply crossover

apply mutation with p_m ; // mutate offspring/children

calculate fitness values; // eval. population

replace current population;

} while termination criteria not satisfied;

end



GA Components – Reproduction

- Reproduction (Mate Selection) consists of
 - **selecting individuals:** apply selection pressure considering the fitness of individuals in the population $\Rightarrow$ e.g., ***roulette wheel selection, tournament selection, rank selection, truncation selection, Boltzmann selection, etc.***
 - Selection pressure means that individuals with better fitness have a higher chance of being selected
 - usually **2 *parents*** (individuals/candidate solutions) are selected using the same method, which will go under the crossover operation

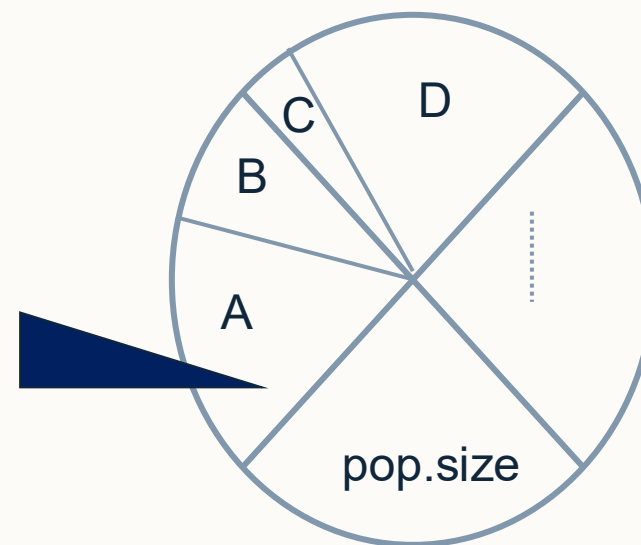
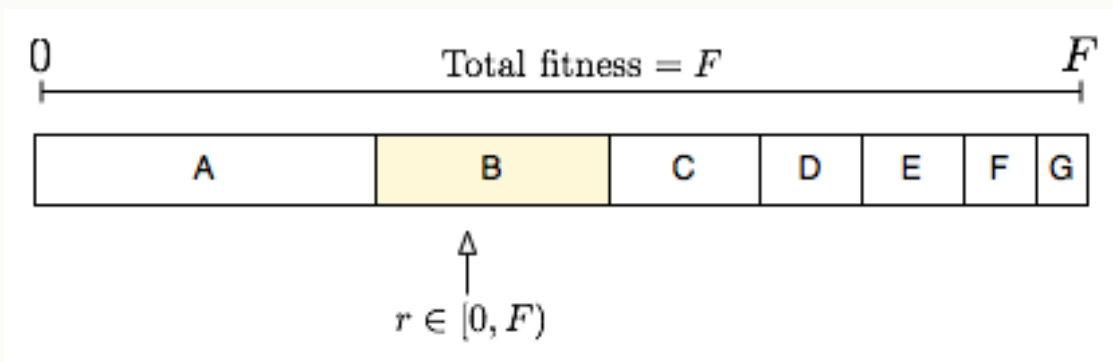


Fitness Proportionate Selection – Roulette Wheel Selection

- Fitness level is used to associate a probability ($prob_i$) of selection with each individual chromosome (i)
- While candidate solutions with a higher fitness will be less likely to be eliminated, there is still a chance that they may be (maximisation problem)
- Expected number of representatives of each individual in the pool is proportional to its fitness (maximisation problem)

maximisation
problem

$$prob_i = \frac{fitness_i}{\sum_j fitness_j}, j = 1 \dots pop.size$$





Roulette Wheel Selection – Minimisation Problem

If you are dealing with a minimisation problem (where a lower score, like **error** or **cost**, is better), the standard formula fails, so the values should be transformed.

- Subtraction Method (Windowing): $fitness_i = cost_{max} - cost_i + \varepsilon$
- Reciprocal Method (Inversion): $fitness_i = 1/(cost_i + \varepsilon)$
- Ranking (Rank-based) Method: Instead of using the raw values, you simply sort the population from best (lowest cost) to worst (highest cost)

<u>Rank</u>	<u>Cost (minimising)</u>	<u>Assigned Fitness</u>
1	0.081 (best)	20
2	3.095	19
...	...	...
20	123.1 (worst)	1



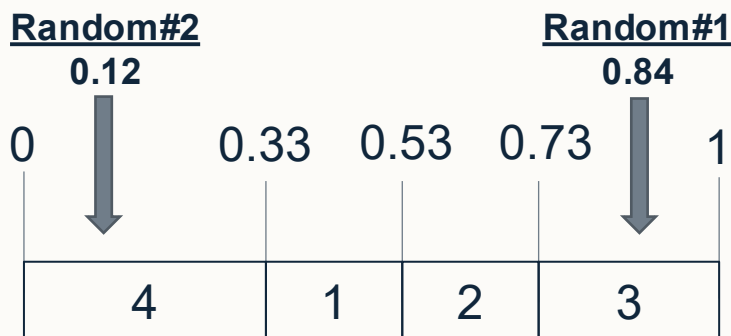
Example: MAX-SAT – Roulette Wheel Selection

$$\text{fitness}_i = \text{cost}_{\max} - \text{cost}_i$$

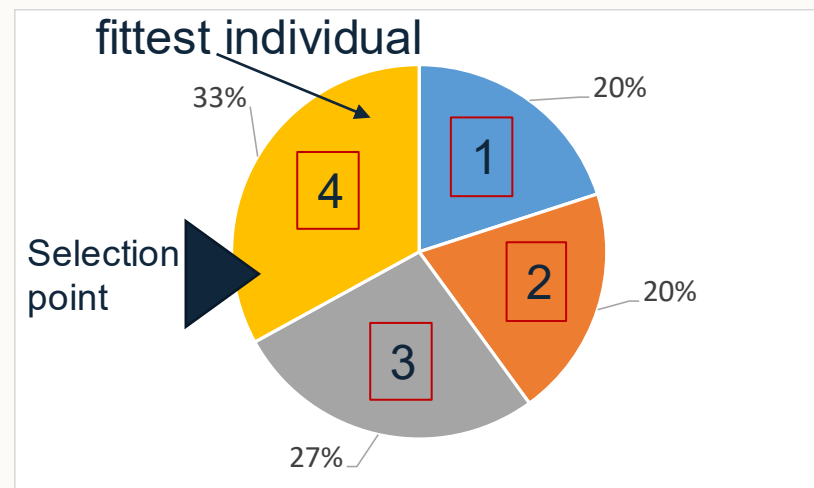
0 1 2 3 4 5
 $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$

i	Chromosome abcdef	Unsat. clauses	Fitness (sat. cl.)	prob _i
1	110000	012345	3 (3)	20%
2	010100	012345	3 (3)	20%
3	001100	012345	2 (4)	27%
4	101110	012345	1 (5)	33%

total:(15)



■ Rotate the wheel



Random#1: 0.84 → Parent#1: 3

Random#2: 0.12 → Parent#2: 4



University of
Nottingham

UK | CHINA | MALAYSIA

Early Module Feedback

Please fill in the early module feedback to include lectures and computing labs – be constructive, responses are anonymous! 😊





Tournament Selection

- This method involves running a number of "tournaments" among randomly chosen individuals (of tour size) selecting the one with best fitness at the end
 - This process is repeated for selecting each parent to be recombined



Example – MAX-SAT: Tournament Selection

tour size = 3, first parent

i	Chromosome abcdef	Violated clauses	Fitness	
1	110000	012345	3	←
2	010100	012345	3	←
3	001100	012345	2	←
4	101110	012345	1	

- Throw a random number between 1 and 4 (population size) for 3 times:
 - [3, 1, 2]
- Tournament selection chooses 3 individuals: #1, #2 and #3 at random, then individual#3 with the fitness of 2 is returned as the first parent



Example – MAX-SAT: Tournament Selection

tour size = 3, second parent

i	Chromosome abcdef	Violated clauses	Fitness	
1	110000	012345	3	←
2	010100	012345	3	
3	001100	012345	2	←
4	101110	012345	1	←

- Throw a random number between 1 and 4 (population size) for 3 times:
 - [4, 1, 3]
- Tournament selection chooses 3 individuals: #1, #3 and #4 at random, then individual#4 with the fitness of 1 is returned as the second parent



Pseudocode of a Generic Genetic Algorithm

Representation

begin

```
generate initial population; // initialise
calculate fitness values;    // evaluate population
do
{
    perform reproduction;    // select parents
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
```

end



Recombination – Crossover

- Selected pairs/mates (*parents*) are recombined to form new individuals (candidate solutions/children/offspring) – exchange of genetic material
- Crossover is applied with a **crossover probability** p_c which in general is chosen close to 1.0
- **One Point Crossover (1PTX)**
 - Generate a random number in $[0,1)$, if it is smaller than a **crossover probability** p_c Then
 - Select a random crossover site in $[1, \text{chromosome length}]$
 - Split individuals at the selected site
 - Exchange segments between pairs to form two new individuals
 - Else
 - Copy the individuals as new individuals



Example – MAX-SAT: 1PTX

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \end{array}$$

Random

number: 2

Randomly determined
crossover point

001100

101110

101100

001110

2. Exchange
the genetic
material

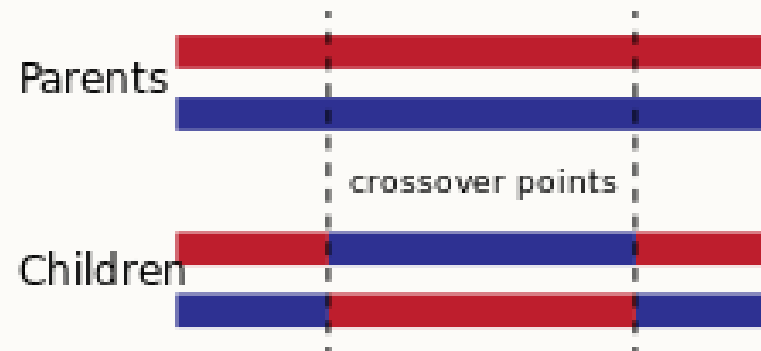
Select the crossover
site by throwing a
random number in...?

new solutions/children/offspring

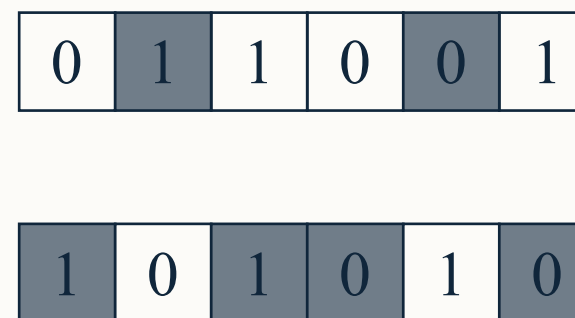
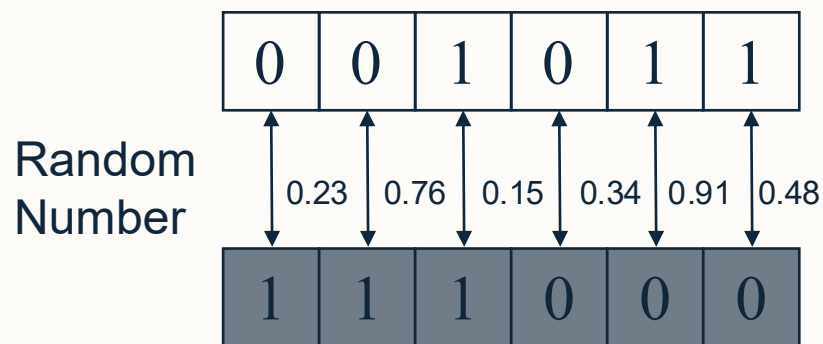


Other Crossover Operators

- 2 Point Crossover (2PTX)
K-point Crossover
Uniform Crossover (UX)



- The uniform crossover considers each bit in the parent strings for exchange with a probability of 0.5.





Pseudocode of a Generic Genetic Algorithm

Representation

begin

```
generate initial population; // initialise
calculate fitness values;    // evaluate population
do
{
    perform reproduction;    // select parents
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
```

end



Mutation

- Any offspring might be exposed to mutation
- Loop through all the alleles of all the individuals one by one, and if that allele is selected for mutation with a given probability p_m , you can either change it by a small amount or replace it with a new value
 - For binary representation mutation corresponds to flipping a selected gene value ($0 \rightarrow 1$, $1 \rightarrow 0$)
- Mutation provides diversity and allows GA to explore different regions of the search space (escaping)
- Mutation rate is typically chosen to be very small (0.001, 0.001).
Choosing p_m as ($1/\text{chromosome length}$) implies on average a single gene will be mutated for an individual.



Example – Mutation

Random numbers:

<0.23 0.76 0.41 0.14 0.91 0.68 0.47 0.16 0.28 0.94 0.78 0.03 ...>

↓ ↓ ↓ ↓ ↓ ↓
0.23 0.76 0.41 0.14 0.91 0.68

1	0	1	0	0	0
---	---	---	---	---	---

↓ ↓ ↓ ↓ ↓ ↓
0.47 0.16 0.28 0.94 0.78 0.03

0	1	1	1	1	1
---	---	---	---	---	---

A loop is performed on each individual
If a random value in $[0,1)$ is $< p_m = 0.17$ ($1/6$), then
the allele value is flipped, otherwise kept same.



Pseudocode of a Generic Genetic Algorithm

Representation

begin

```
generate initial population; // initialise
calculate fitness values;    // evaluate population
do
{
    perform reproduction;    // select parents
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
```

end



Replacement Strategy

- There are variety of strategies for replacing the old population (generation) by the new (offspring) population to form the next generation
- **Generation gap (α)** controls the fraction of the population to be replaced in each generation, where $\alpha \in [1/N, 1.0]$
 - Number of offspring produced at each generation is $g = \alpha * N$
- **(Trans-)Generational GA** ($g > 2$, that is $\alpha > 2/N$)
 N individuals produce αN offspring, so $(N + \alpha N) \rightarrow N$
 αN replaces worst αN of N
largest *generation gap* where $\alpha = 1.0$ yields $g = N$.
GA relies on improvement of average objective values from one population to another
It is always a good idea not to loose the best solution found so far.
sort $(N + \alpha N)$ and choose the N best (elitism)



Replacement Strategy

- ***Steady-State GA*** ($g=2$, that is $\alpha=2/N$)
 - Two offspring replace two individuals from the old generation.
 - Method#1: two offspring replace two parents
 - Method#2: two offspring replace worst two of the population
 - Method#3: best two of (parents and offspring) replace two parents (elitism)
 - Method#4: best two of (parents and offspring) replace worst two of the population (strong elitism)

What methods can be used?



Example – Transgenerational GA Replacement (no elitism)

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \end{array}$$

i	Chromosome abcdef	Unsat. clauses	Fitness
1	100100	012345	3
2	010100	012345	3
3	101000	012345	0
4	011111	012345	0

New Population/
Generation

Form the new
generation by

←
copying
offspring
onto the old
population,
forming the
new
population

($g=N=4$)

i	Chromosome abcdef	Unsat. clauses	Fitness
1	100100	012345	3
2	010100	012345	3
3	101000	012345	0
4	011111	012345	0

Offspring

i	Chromosome abcdef	Unsat. clauses	Fitness
1	110000	012345	3
2	010100	012345	3
3	001100	012345	2
4	101110	012345	1

Old Population



Example – TGA Replacement (with elitism)

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \end{array}$$

Chromosome		Unsat.	Fitness
i	abcdef	clauses	
1	001100	012345	2
2	101110	012345	1
3	101000	012345	0
4	011111	012345	0

New Population/
Generation

Form the new
generation by

←
Selecting the
best 4
individuals
among both
old population
and offspring

($g=N=4$)

Chromosome		Unsat.	Fitness
i	abcdef	clauses	
1	100100	012345	3
2	010100	012345	3
3	101000	012345	0
4	011111	012345	0

Offspring

Chromosome		Unsat.	Fitness
i	abcdef	clauses	
1	110000	012345	3
2	010100	012345	3
3	001100	012345	2
4	101110	012345	1

Old Population



Example – A Steady State GA (with elitism)

$$\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \\ (a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e) \end{matrix}$$

i	Chromosome		Unsat. clauses	Fitness
	abcdef			
1	010100	012345	3	
2	101000	012345	0	
3	001100	012345	2	
4	101110	012345	1	

New Population/
Generation

Next
generation
←
Using
Method#2:
Replace the
worst 2 of the
population
with 2
offspring
($g=2$)

i	Chromosome		Unsat. clauses	Fitness
	abcdef			
1	010100	012345	3	
2	101000	012345	0	

Offspring

i	Chromosome		Unsat. clauses	Fitness
	abcdef			
1	110000	012345	3	
2	010100	012345	3	
3	001100	012345	2	
4	101110	012345	1	

Old Population



Pseudocode of a Generic Genetic Algorithm

Representation

begin

```
generate initial population; // initialise
calculate fitness values;    // evaluate population
do
{
    perform reproduction;    // select parents
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
```

end



Termination Criteria

- The evolution (main loop) continues until a termination criteria is met, possibly until:
 - A predefined maximum number of generations is exceeded
 - A goal is reached, for example:
 - Expected fitness is achieved
 - Population converges
 - Best fitness does not change for a while
 - A condition is satisfied depending on a combination of above



Convergence

- Defined as the progression towards uniformity (individuals become alike)
 - *Gene convergence*: a location on a chromosome is converged when 95% of the individuals have the same gene value for that location
 - *Population (Genotypic) convergence*: a population is converged when all the genes have converged (all individuals are alike – they might have different fitness)
 - *Phenotypic Convergence*: average fitness of the population approaches to the best individual in the population (all individuals have the same fitness)



Key Features of EAs

- Population based search approaches
 - Be independent of initial starting point(s)
 - Start search from many points in the search space
 - Conduct search in parallel over the search space
 - implicit parallelism
- Avoid converging to local optima
- Balances exploration and exploitation?
- May be used together with other approaches (hybrids)



Memetic Algorithms (MAs)



Memetic Algorithms (MAs)

- Meme (Richard Dawkins): contagious piece of information
- Memes are similar to local refinement/local search
- Gene vs meme: Memes can change, evolve using rules and time scales other than the traditional genetic ones
- MAs aim to improve GAs by embedding local search



Memetic Algorithms (MAs) II

- MAs make use of exploration capabilities of GAs and exploitation capabilities of local search
 - MAs have an explicit mechanism to balance exploitation and exploration
- Memetic Algorithms shown to be much faster and more accurate than GAs on some problems, and are the “state of the art” on many problems



Pseudocode of a Generic Genetic Algorithm

Representation

begin

```
generate initial population; // initialise
calculate fitness values;    // evaluate population
do
{
    perform reproduction;    // select parents
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
```

end



Pseudocode of a Generic Memetic Algorithm

Representation

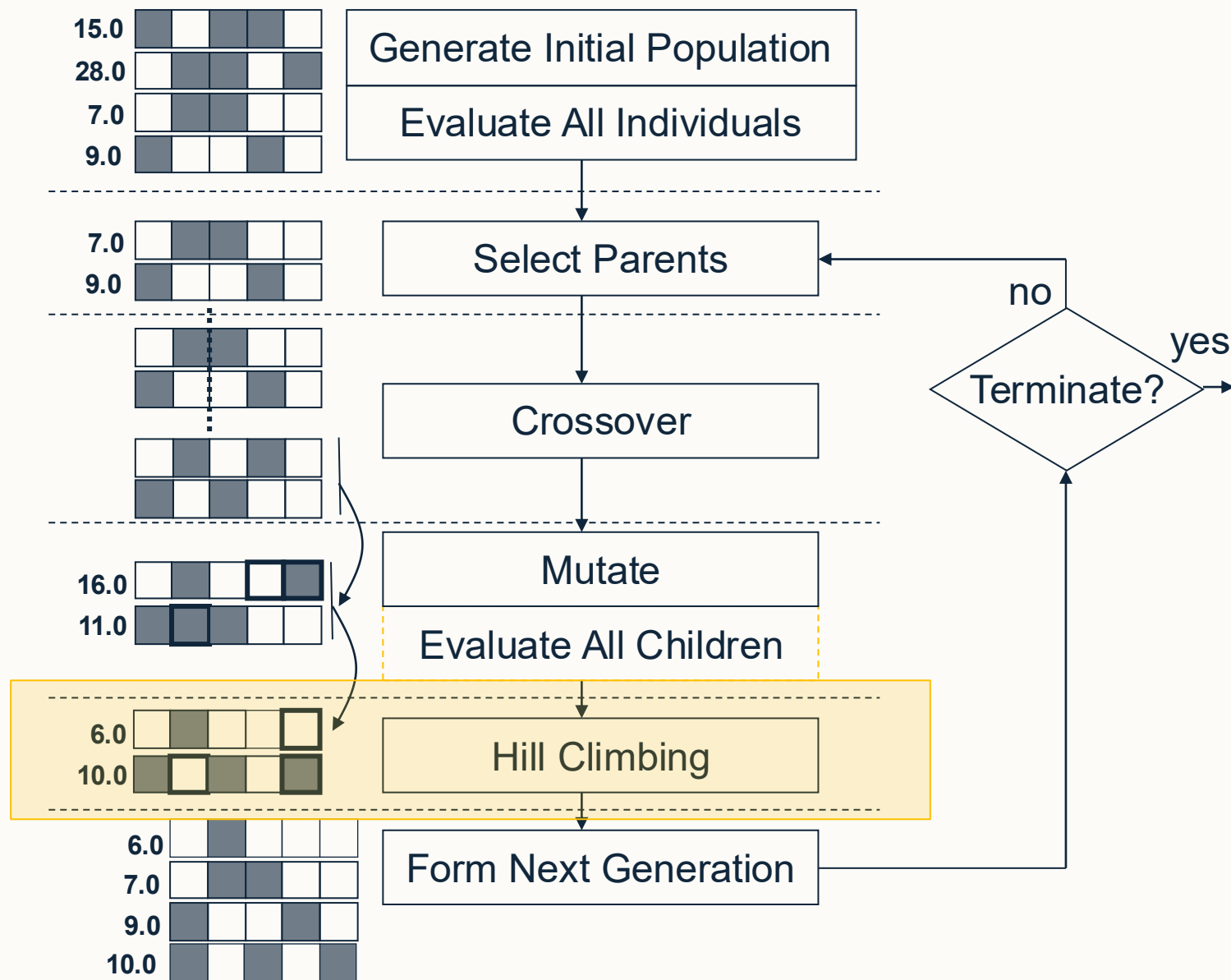
begin

```
generate initial population; // initialise
calculate fitness values;    // evaluate population
do
{
    perform reproduction;    // select parents
    recombine pairs with  $p_c$ ; // apply crossover
    apply mutation with  $p_m$ ; // mutate offspring/children
    apply local search;      // hill climbing on offspring
    calculate fitness values; // eval. population
    replace current population;
} while termination criteria not satisfied;
```

end



A Generic Memetic Algorithm (MA)





Pseudocode of a Generic Memetic Algorithm

Representation

Moscato, P.: On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms, Caltech Concurrent Computation Program Report 826, California Institute of Technology (1989)

begin

```
→ generate initial population; // initialise
   calculate fitness values;    // evaluate population
   do
   {
   → perform reproduction;      // select parents
   → recombine pairs with  $p_c$ ; // apply crossover
     apply mutation with  $p_m$ ; // mutate offspring/children
     apply hill climbing;      // local search on offspring
     calculate fitness values; // eval. population
     replace current population;
   } while termination criteria not satisfied;
end
```



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT

- *Representation*: Bit string of length N (truth assignment for each variable)
- *Initialisation*: Randomly generate initial population, population size= N
- *Fitness function*: ($\#$ of clauses – C); $\#$ of unsatisfied clauses
- *Mate selection*: Tournament selection with a tour size of 2
- *Crossover*: 1PTX, crossover probability = 0.99
- *Mutation*: random bit-flip, mutation probability = $1/N$
- **Hill Climbing: Davis's Bit Hill Climbing**
- *Replacement*: Steady State GA, best two of (parents and offspring) replaces the worst two individuals in the population (strong elitism)



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT: Initilisation

Initial
Population
110000
010100
001100
101110

Genetic Algorithm Steps:

- Initialisation:
 - Population is already given, so we skip random initialisation.

Memetic Algorithm Steps:

- Initialisation:
 - Population is already given, so we skip random initialisation.



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT: Evaluation

Initial
Population
110000
010100
001100
101110

Genetic Algorithm Steps:

- Fitness Evaluation:
 - 110000 (Fitness: 3)
 - 010100 (Fitness: 6)
 - 001100 (Fitness: 5)
 - 101110 (Fitness: 5)

Memetic Algorithm Steps:

- Fitness Evaluation:
 - 110000 (Fitness: 3)
 - 010100 (Fitness: 6)
 - 001100 (Fitness: 5)
 - 101110 (Fitness: 5)



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT: Parent Selection

Initial
Population
110000
010100
001100
101110

Genetic Algorithm Steps:

- Selection:
 - Use tournament selection (size 2) to choose parents.
 - Example: Select (010100, Fitness 6) and (001100, Fitness 5)

Memetic Algorithm Steps:

- Selection:
 - Use tournament selection (size 2) to choose parents.
 - Example: Select (010100, Fitness 6) and (001100, Fitness 5)



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT

Initial
Population
110000
010100
001100
101110

Genetic Algorithm Steps:

- Crossover:
 - Use single-point crossover.

Parents	After Crossover
010 100	010100
001 100	001100

Memetic Algorithm Steps:

- Crossover:
 - Use single-point crossover.

Parents	After Crossover
010 100	010100
001 100	001100



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT: Mutation

Initial
Population
110000
010100
001100
101110

Genetic Algorithm Steps:

- Mutation:
 - Apply bit-flip mutation with a low probability (e.g., 1%).
 - Example: 010100 -> 011100 (3rd bit flipped)
 - 001100 remains unchanged

Memetic Algorithm Steps:

- Mutation:
 - Apply bit-flip mutation with a low probability (e.g., 1%).
 - Example: 010100 -> 011100 (3rd bit flipped)
 - 001100 remains unchanged



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT: Initialisation

Initial

Population

110000

010100

001100

101110

Genetic Algorithm Steps:

- Fitness Evaluation of Offspring:

- 011100 (Fitness: 5)

- 001100 (Fitness: 5)

Memetic Algorithm Steps:

- Local Search (Hill Climbing):

- Apply to each offspring.

- For 011100:

- 011100 (Fitness: 5)

- 111100 (F:4, revert) -> 011100

- 001100 (F:5, no change) -> 011100

- 010100 (F:6, keep) -> 010100

- 010000 (F:4, revert) -> 010100

- 010110 (F:5, revert) -> 010100

- 010101 (F:5, revert) -> 010100

- Final: 010100 (Fitness: 6)

- For 001100:

- Start: 001100 (Fitness: 5)

- (Similar process, no improvement found)

- Final: 001100 (Fitness: 5)



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT: Local Search

Initial
Population
110000
010100
001100
101110

Genetic Algorithm Steps:

- Fitness Evaluation of Offspring:
 - 011100 (Fitness: 5)
 - 001100 (Fitness: 5)

Memetic Algorithm Steps:

- Local Search (Hill Climbing):
 - 010100 (Fitness: 6)
 - 001100 ((Fitness: 5)



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT: Replacement

Initial
Population
110000
010100
001100
101110

Genetic Algorithm Steps:

- Replace the two worst individuals in the population.
- New population:
 - 010100 (Fitness: 6)
Original best
 - 011100 (Fitness: 5)
New offspring
 - 001100 (Fitness: 5)
New offspring
 - 101110 (Fitness: 5)
Retained from original population

Memetic Algorithm Steps:

- Replace the two worst individuals in the population.
- New population:
 - 010100 (Fitness: 6)
Original best
 - **010100 (Fitness: 6)**
Improved offspring
 - 001100 (Fitness: 5)
Unchanged offspring
 - 101110 (Fitness: 5)
Retained from original population



Summary

- Evolutionary Algorithms
- Genetic Algorithms (GAs)
- Memetic Algorithms (MAs)





Thank you

Any questions?